

JavaScript OOP Masterclass

Curated MAANG Interview Guide • Fundamentals to Advanced Object-Oriented Patterns

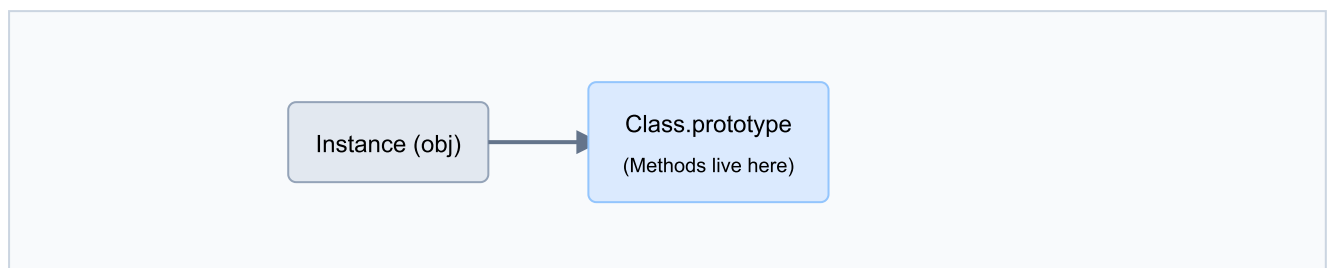
I. Fundamentals

1. What is a JavaScript class, and how does it differ internally from a traditional class in Java or C++?

In Java or C++, a class is a static blueprint compiled into memory; objects are rigid instances of that blueprint. In JavaScript, a `class` is primarily syntactic sugar over prototypal inheritance. Internally, a JS class is just a function (the constructor), and its methods are assigned to the constructor function's prototype object. Because JS is dynamic, you can mutate a class's prototype at runtime, immediately affecting all instances—a concept impossible in static classical languages.

2. What happens when you instantiate a class using `new`?

When `new MyClass()` is called, the engine performs four steps: 1. It creates a new, empty plain JavaScript object. 2. It links the new object's internal `[[Prototype]]` to `MyClass.prototype`. 3. It executes the constructor function with `this` bound to the newly created object, initializing properties. 4. It returns the new object (unless the constructor explicitly returns a non-primitive object).



3. What is the difference between an instance method and a static method?

An **instance method** is defined on the class's prototype (e.g., `User.prototype.login`). It is shared among all instances and accessed via instances (`user.login()`). A **static method** is defined directly on the constructor function object itself (e.g., `User.findById`). It cannot be called on instances; it is accessed via the class name.

8. How does `instanceof` work, and why can it produce unexpected results across browser realms?

The `instanceof` operator traverses the object's prototype chain, checking if the constructor's prototype property appears anywhere in it. It can fail across execution realms (like iframes or web workers) because an `Array` created in an iframe has a different global `Array` constructor than the main window, causing `arr instanceof window.Array` to evaluate to `false`.

II. Intermediate & Advanced Concepts

Intermediate Q2: Why must a derived constructor call `super()` before accessing `this`?

In a derived class (one that uses `extends`), the derived constructor does not create the `this` object itself. Instead, it delegates object creation to the parent class's constructor. Calling `super()` invokes the parent constructor, which creates the instance and binds it to `this`. Accessing `this` before `super()` throws a `ReferenceError` because the instance literally does not exist yet in the child's scope.

Intermediate Q5: What is the difference between a class method and an arrow-function class field?

A standard class method (`greet() {}`) is added to the prototype, meaning it is shared in memory across all instances. It relies on dynamic `this` binding based on how it is invoked. An arrow-function class field (`greet = () => {}`) is assigned to *every individual instance* during construction. It permanently binds `this` lexically to the instance, making it useful for passing as callbacks, but it consumes more memory since a new function is created per instance.

Feature	Class Method <code>method() {}</code>	Arrow Function Field <code>method = () => {}</code>
Location	Class prototype	Individual Object Instance
Memory	Highly efficient (shared)	Less efficient (duplicated)
<code>`this`</code> Binding	Dynamic (caller-dependent)	Lexical (permanently bound to instance)

Advanced Q1: Explain composition versus inheritance using a real-world example.

Inheritance defines an "is-a" relationship (e.g., `EmailNotification` extends `Notification`). This often leads to the "fragile base class" problem and deep, inflexible hierarchies when trying to combine features (e.g., an `EncryptedSmsNotification`). Composition defines a "has-a" relationship, building objects by combining modular behaviors. Instead of subclassing, a `NotificationService` class can be composed of interchangeable `Transport` (Email/SMS) and `Formatter` (Plain/Encrypted) modules, allowing maximum flexibility at runtime without deeply nested hierarchies.

Deep-Dive Q5: How do JavaScript engines optimize object shapes?

V8 and SpiderMonkey use "Hidden Classes" (or Shapes) to optimize property access. If multiple objects are initialized with the exact same properties in the exact same order, they share a hidden class, allowing the engine to compile property lookups into fast, direct memory offsets. Inconsistent initialization (conditionally adding fields) or mutating the prototype dynamically forces the engine to create new hidden classes or de-optimize to slower dictionary lookups.

III. Output-Based Code Challenges

33. What is the output, and why?

```
class Parent { value = 10; constructor() { console.log(this.value); } } class Child extends Parent { value = 20; } new Child();
```

Output: 10

Why: When `new Child()` is called, the default child constructor automatically calls `super()`. In JavaScript, class fields are initialized sequentially. The `Parent` class initializes `value = 10`, then its constructor body executes, logging 10. Only *after* `super()` finishes does the `Child` class initialize its own fields, overwriting `value` to 20. The log happens before the overwrite.

34. What is the output, and why?

```
class Parent { greet() { return "Parent"; } } class Child extends Parent { greet() { return super.greet() + " Child"; } } console.log(new Child().greet());
```

Output: "Parent Child"

Why: The `Child` instance calls its own `greet()` method. Inside it, `super.greet()` statically looks up the `greet` method on `Parent.prototype` and executes it with `this` bound to the current child instance. It returns "Parent", which is concatenated with " Child".

35. What is the output, and why?

```
class Counter { count = 0; increment = () => ++this.count; } const a = new Counter(); const b = new Counter(); console.log(a.increment === b.increment); console.log(a.increment()); console.log(b.count);
```

Output: false, 1, 0

Why: Because `increment` is an arrow-function class field, it is created as a unique function on each instance rather than shared on the prototype. Thus, `a.increment === b.increment` is false. Calling `a.increment()` updates `a`'s count to 1. `b`'s state is completely independent, so `b.count` remains 0.

III. Output-Based Challenges (Continued)

36. What happens when this code executes?

```
class Example { constructor() { this.value = 1; return { value: 2 }; } } const example = new Example(); console.log(example.value); console.log(example instanceof Example);
```

Output: 2, false

Why: By design, if a JavaScript constructor returns a non-primitive object, that object overrides the default instantiated instance. The engine discards the `this` object (which had `value = 1`) and returns `{ value: 2 }`. Because this returned object is a plain object literal, its prototype is `Object.prototype`, not `Example.prototype`, making the `instanceof` check false.

37. What happens when this code executes, and why?

```
class Parent { #value = 10; } class Child extends Parent { getValue() { return this.#value; } }
```

Output: `SyntaxError` (Private field '#value' must be declared in an enclosing class).

Why: Private fields (prefixed with #) are strictly scoped to the exact class definition block where they are declared. Subclasses do not have access to their parent's private fields. This provides hard encapsulation at the language level.

40. What is the output, and why?

```
class A {} class B extends A {} const obj = new B(); console.log(obj instanceof B); console.log(obj instanceof A); Object.setPrototypeOf(obj, {}); console.log(obj instanceof B); console.log(obj instanceof A);
```

Output: true, true, false, false

Why: Initially, `obj`'s prototype chain is: `obj -> B.prototype -> A.prototype -> Object.prototype`. Thus, both initial `instanceof` checks pass. However, `Object.setPrototypeOf(obj, {})` forcibly mutates `obj`'s prototype, changing the chain to: `obj -> {} -> Object.prototype`. Since `B.prototype` and `A.prototype` are no longer in the chain, subsequent `instanceof` checks fail. This illustrates why `instanceof` is dynamic and can be manipulated.